

学号： 2104230414

沈阳建筑大学

上机报告

课程名称： 算法设计与分析

项目名称： 贪心算法

2023年秋季学期

上机时间：第 4 周 星期 二 第 3 ~ 4 节

学生姓名： 张清晨 专业班级： 计算机 2101 班

指导教师： 刘俊岭 成 绩：

项目名称	动态规划(二)		学时	2
日期：2023 年 9 月 19 日		第 4 教学周		
硬件及软件				
名 称		版本、型号		
Visual C++		6.0 及以上		
上机目的与要求： 1. 问题描述，题目定义。 2. 算法实例，实例演示程序过程。 3. 程序代码，完整可运行程序，并加注释。 4. 程序验证，要求有不同的输入、输出，并截图。 5. 算法分析，时间复杂度与空间复杂度。 本次任务： 实验 3-7：编辑距离 实验 4-1：部分背包问题(贪心算法) 实验 4-2：活动安排(贪心算法) 实验 4-3：最大子段和(贪心算法)				

实验 3-7: 编辑距离

1. 问题描述, 题目定义。

给定两个序列 X 和 Y, 求两个序列编辑距离。

示例 1:

输入: X: "ABCADBCA" Y: "BACDBDA"

输出: 5

2. 算法实例, 实例演示程序过程。

输入:

	1	2	3	4	5	6	7
s	A	B	C	B	D	A	B
t	B	D	C	A	B	A	

递推公式:

$$D[i,j] = \min \begin{cases} D[i-1,j]+1 & \text{删除} \\ D[i,j-1]+1 & \text{插入} \\ D[i-1,j-1] + \begin{cases} 0, & \text{if } s[i]=t[j] \\ 1, & \text{if } s[i]\neq t[j] \end{cases} & \text{替换} \end{cases}$$

	j=0	1	2	3	4	5	6		j=0	1	2	3	4	5	6
i=0	0	1	2	3	4	5	6	i=0		L	L	L	L	L	L
1	1	1	2	3	3	4	5	1	U	LU	LU	LU	LU	L	LU
2	2	1	2	3	4	3	4	2	U	LU	LU	LU	LU	LU	L
3	3	2	2	2	3	4	4	3	U	U	LU	LU	L	L	LU
4	4	3	3	3	4	4	4	4	U	LU	LU	LU	LU	LU	L
5	5	4	3	4	4	4	4	5	U	U	LU	LU	LU	LU	LU
6	6	5	4	4	4	5	4	6	U	U	U	LU	LU	LU	LU
7	7	6	5	5	5	4	5	7	U	LU	U	LU	LU	LU	L

3. 程序代码，完整可运行程序，并加注释。

```
/*编辑距离*/
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

class Solution {
public:
    int min3(int a, int b, int c){
        int m=a;
        if (m > b)
            m = b;
        if (m > c)
            m = c;
        return m;
    }
    vector<vector<int>> minEditDistance(string s, string t, vector<vector<int>> dp) {
        vector<vector<int>> rec(s.size() + 1, vector<int>(t.size() + 1, 0));
        //初始化两个数组
        for (int i = 0; i <= s.size(); i++) dp[i][0] = i ;
        for (int j = 0; j <= t.size(); j++) dp[0][j] = j ;
        //rec 数组 取 1 表示 U， dp[i][j]由 dp[i-1][j]获得，即上侧 U。
        //rec 数组 取 -1 表示 L， dp[i][j]由 dp[i][j-1]获得，即左侧 L。
        //rec 数组 取 0 表示 LU， dp[i][j]由 dp[i-1][j-1]获得，即左上 LU。
        rec[0][0] = 0;
        for (int i = 1; i <= s.size(); i++) rec[i][0] = 1 ;
        for (int j = 1; j <= t.size(); j++) rec[0][j] = -1 ;

        for (int i = 1; i < s.size(); i++) {
            for (int j = 1; j < t.size(); j++) {
                int c = 0;
                if (s[i] != t[j]) c = 1;
                int replaceO = dp[i - 1][j - 1] + c;
                int deleteO = dp[i - 1][j] + 1;
                int insertO = dp[i][j - 1] + 1;
                //替换操作
                if (replaceO == min3(deleteO, insertO, replaceO)){
                    dp[i][j] = dp[i - 1][j - 1] + c ;
                    rec[i][j] = 0 ;    //左上 LU
                }
                //插入操作
```

```

        else if (insertO == min3(deleteO, insertO, replaceO)){
            dp[i][j] = dp[i][j - 1] + 1;
            rec[i][j] = -1 ;    //左 L
        }
        //删除操作
        else{
            dp[i][j] = dp[i - 1][j] + 1;
            rec[i][j] = 1 ;    //上 U
        }
    }
}
//打印 dp 数组
cout << "dp 数组:" << endl;
for (int i = 0; i < s.size() ; i++){
    for (int j = 0; j < t.size() ; j++){
        cout << "dp[i][j] = " << dp[i][j] << ", ";
    }
    cout << endl;
}

return rec;
}
//追踪最优解操作序列，递归函数后正序输出操作序列
void PrintMED(vector<vector<int>> rec, string s, string t, int i, int j)
{
    if (i == 0 && j == 0)
        return ;
    if (rec[i][j] == 0)
    {
        PrintMED(rec, s, t, i - 1 , j - 1 );//
        if (s[i] == t[j])
            printf("无操作\n");//
        else
            printf("用%c 替换%c\n", t[j], s[i]);//
    }
    else if (rec[i][j] == 1){
        PrintMED(rec, s, t, i - 1 , j );
        printf("删除%c\n", s[i] );//
    }

    else{
        PrintMED(rec, s,t, i , j - 1 );
        printf("插入%c\n", s[j]);//
    }
}

};

```

```

int main() {

    Solution solution;
    string s = " ABCBDAB";
    string t = " BDCABA";
    vector<vector<int>> dp(s.size() + 1, vector<int>(t.size() + 1, 0));
    vector<vector<int>> rec(s.size() + 1, vector<int>(t.size() + 1, 0));
    rec = solution.minEditDistance(s, t, dp);
    cout << endl;
    //打印追踪数组 rec
    cout << "rec 数组:" << endl;
    for (int i = 0; i < s.size(); i++){
        for (int j = 0; j < t.size(); j++){
            cout << "rec[i][j] = " << rec[i][j] << ",";
        }
        cout << endl;
    }

    solution.PrintMED(rec, s, t, s.size()-1, t.size()-1);

    system("PAUSE");
    return 0;
}

```

4. 程序验证，要求有不同的输入、输出，并截图。

```

string s = " ABCBDAB";
string t = " BDCABA";

```

The screenshot shows the output of the program in a window titled "E:\算法设计与分析\4\实验3-7: 编辑距离.exe". The output displays the DP table and the sequence of edit operations for transforming the string "ABCBDAB" into "BDCABA".

DP Table Output:

```

dp数组:
dp[i][j] = 0, dp[i][j] = 1, dp[i][j] = 2, dp[i][j] = 3, dp[i][j] = 4, dp[i][j] = 5, dp[i][j] = 6,
dp[i][j] = 1, dp[i][j] = 1, dp[i][j] = 2, dp[i][j] = 3, dp[i][j] = 3, dp[i][j] = 4, dp[i][j] = 5,
dp[i][j] = 2, dp[i][j] = 1, dp[i][j] = 2, dp[i][j] = 3, dp[i][j] = 4, dp[i][j] = 3, dp[i][j] = 4,
dp[i][j] = 3, dp[i][j] = 2, dp[i][j] = 2, dp[i][j] = 3, dp[i][j] = 4, dp[i][j] = 4, dp[i][j] = 4,
dp[i][j] = 4, dp[i][j] = 3, dp[i][j] = 3, dp[i][j] = 3, dp[i][j] = 3, dp[i][j] = 3, dp[i][j] = 4,
dp[i][j] = 5, dp[i][j] = 4, dp[i][j] = 3, dp[i][j] = 4, dp[i][j] = 4, dp[i][j] = 4, dp[i][j] = 4,
dp[i][j] = 6, dp[i][j] = 5, dp[i][j] = 4, dp[i][j] = 4, dp[i][j] = 5, dp[i][j] = 4, dp[i][j] = 4,
dp[i][j] = 7, dp[i][j] = 6, dp[i][j] = 5, dp[i][j] = 5, dp[i][j] = 5, dp[i][j] = 4, dp[i][j] = 5,

```

rec Array Output:

```

rec数组:
rec[i][j] = 0, rec[i][j] = -1, rec[i][j] = -1, rec[i][j] = -1, rec[i][j] = -1, rec[i][j] = -1, rec[i][j] = -1,
rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = -1, rec[i][j] = 0,
rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = -1,
rec[i][j] = 1, rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = -1, rec[i][j] = -1, rec[i][j] = 0,
rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = -1,
rec[i][j] = 1, rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0,
rec[i][j] = 1, rec[i][j] = 1, rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0,
rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = -1,

```

Edit Operations:

```

删除A
无操作
删除C
用D替换B
用C替换D
无操作
无操作
插入A
请按任意键继续. . .

```

```
string s = " ABCDAB";
string t = " DCABA";
```

```
E:\算法设计与分析\4\实验3-7: 编辑距离.exe
dp数组:
dp[i][j] = 0, dp[i][j] = 1, dp[i][j] = 2, dp[i][j] = 3, dp[i][j] = 4, dp[i][j] = 5,
dp[i][j] = 1, dp[i][j] = 1, dp[i][j] = 2, dp[i][j] = 2, dp[i][j] = 3, dp[i][j] = 4,
dp[i][j] = 2, dp[i][j] = 2, dp[i][j] = 2, dp[i][j] = 3, dp[i][j] = 2, dp[i][j] = 3,
dp[i][j] = 3, dp[i][j] = 3, dp[i][j] = 2, dp[i][j] = 3, dp[i][j] = 3, dp[i][j] = 3,
dp[i][j] = 4, dp[i][j] = 3, dp[i][j] = 3, dp[i][j] = 3, dp[i][j] = 4, dp[i][j] = 4,
dp[i][j] = 5, dp[i][j] = 4, dp[i][j] = 4, dp[i][j] = 3, dp[i][j] = 4, dp[i][j] = 4,
dp[i][j] = 6, dp[i][j] = 5, dp[i][j] = 5, dp[i][j] = 4, dp[i][j] = 3, dp[i][j] = 4,

rec数组:
rec[i][j] = 0, rec[i][j] = -1, rec[i][j] = -1, rec[i][j] = -1, rec[i][j] = -1, rec[i][j] = -1,
rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = -1, rec[i][j] = 0,
rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 1, rec[i][j] = 0,
rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0,
rec[i][j] = 1, rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0, rec[i][j] = 0,
rec[i][j] = 1, rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = 1, rec[i][j] = 0, rec[i][j] = -1,

删除A
用D替换B
无操作
删除D
无操作
无操作
插入A
请按任意键继续. . .
```

5. 算法分析，时间复杂度与空间复杂度。

该算法使用了动态规划，是动态规划类算法。使用了二维的 dp 数组以及二维的 rec 数组，所以空间复杂度为 $n*m$ ，时间复杂度上嵌套使用了两个 for 循环，所以也是 $n*m$

时间复杂度: $O(n*m)$

空间复杂度: $O(n*m)$

实验 4-1：部分背包问题(贪心算法)

1. 问题描述，题目定义。

在选择物品 i 装入背包时，可以选择物品 i 的一部分，而不一定要全部装入背包， $1 \leq i \leq n$ 。

示例 1：

输入：

//假设已按单位价值降序排序

```
float profit[5] = { 0, 10, 16, 6, 1};
```

```
float weight[5] = { 0, 3, 5, 2, 1};
```

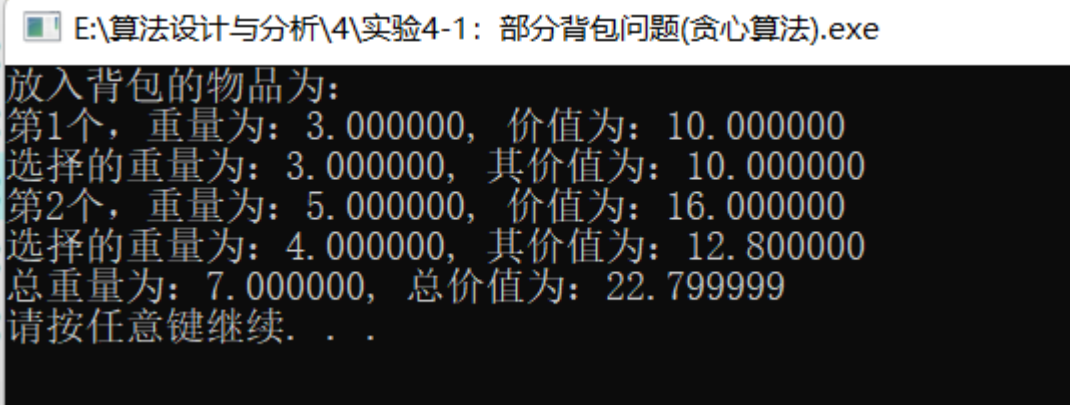
输出：

2. 算法实例，实例演示程序过程。

输入：物品已经按单位价值降序排序，既 $A > B > C > D$

items	A: 0	B: 1	C: 2	D: 3
profit	10	16	6	1
weight	3	5	2	1

输出：



E:\算法设计与分析\4\实验4-1：部分背包问题(贪心算法).exe

放入背包的物品为：
第1个，重量为：3.000000，价值为：10.000000
选择的重量为：3.000000，其价值为：10.000000
第2个，重量为：5.000000，价值为：16.000000
选择的重量为：4.000000，其价值为：12.800000
总重量为：7.000000，总价值为：22.799999
请按任意键继续. . .

3. 程序代码，完整可运行程序，并加注释。

```
/*部分背包问题*/  
#define _CRT_SECURE_NO_WARNINGS  
#include <stdio.h>  
#include "stdlib.h"  
#include <iostream>  
#include <vector>
```



```

#include <algorithm>

using namespace std;

void knapsack(int n, float m, float profit[], float weight[], float *x);

int main(){
    //假设已按单位价值降序排序
    float profit[5] = { 0, 10, 16, 6, 1};
    float weight[5] = { 0, 3, 5, 2, 1};
    int n = 4;
    float c = 7;

    float x[5] = { 0, 0, 0, 0, 0 };

    float sumWeight = 0, sumProfit = 0;

    knapsack(n, c, profit, weight, x);

    printf("放入背包的物品为: \n");
    for (int i = 1; i <= n; i++)
    {

        if (x[i] != 0){
            printf("第%d 个, 重量为: %f, 价值为: %f\n", i, weight[i], profit[i]);
            printf("选择的重量为: %f, 其价值为: %f\n", x[i], x[i]*(profit[i]/weight[i]));
            sumWeight += x[i];
            sumProfit += x[i]*(profit[i]/weight[i]);
        }
        //printf("%f ", x[i]);
    }
    printf("总重量为: %f, 总价值为: %f\n", sumWeight, sumProfit);

    system("PAUSE");
    return 0;
}

void knapsack(int n, float m, float profit[], float weight[], float *x)
{
    int i=0;

    float c = m;

    for (i = 1; i <= n; i++)
    {

```

```

        if (weight[i] > c)
            break;
        x[i] = weight[i];
        c -= weight[i];
    }
    if (i <= n)
        x[i] = c;
        c = 0;
}

```

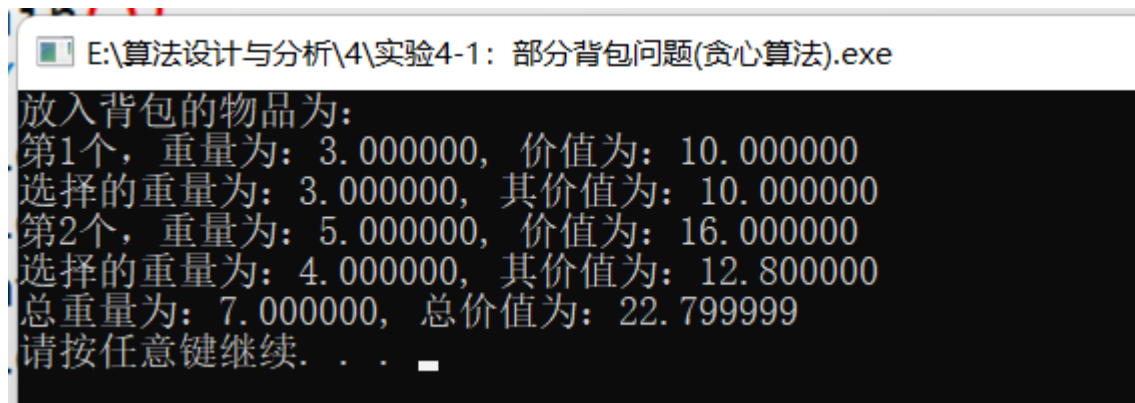
4. 程序验证，要求有不同的输入、输出，并截图。

// 假设已按单位价值降序排序

```

float profit[5] = { 0, 10, 16, 6, 1};
float weight[5] = { 0, 3, 5, 2, 1};
int n = 4;
float c = 7;

```



// 假设已按单位价值降序排序

```

float profit[5] = { 0, 10, 16, 6, 1};
float weight[5] = { 0, 3, 5, 2, 1};
int n = 4;
float c = 9;

```

E:\算法设计与分析\4\实验4-1: 部分背包问题(贪心算法).exe

```
放入背包的物品为:
第1个, 重量为: 3.000000, 价值为: 10.000000
选择的重量为: 3.000000, 其价值为: 10.000000
第2个, 重量为: 5.000000, 价值为: 16.000000
选择的重量为: 5.000000, 其价值为: 16.000000
第3个, 重量为: 2.000000, 价值为: 6.000000
选择的重量为: 1.000000, 其价值为: 3.000000
总重量为: 9.000000, 总价值为: 29.000000
请按任意键继续. . .
```

5. 算法分析, 时间复杂度与空间复杂度。

本算法的时间复杂度应由两部分组成, 一部分是对物品按单位价值进行排序的时间复杂度, 这部分的时间复杂度为 $O(n \log n)$, 另外一部分是按照贪心策略进行求最优解, 这部分的时间复杂度为 $O(n)$ 。但是题目中默认给出的物品已经按照单位价值进行降序排序了, 所以本题的算法的时间复杂度为 $O(n)$ 。而空间复杂度, 本次算法只用了三个长度为 n 的一维数组, 所以空间复杂度为 $O(n)$ 。所以

- 时间复杂度: $O(n)$
- 空间复杂度: $O(n)$

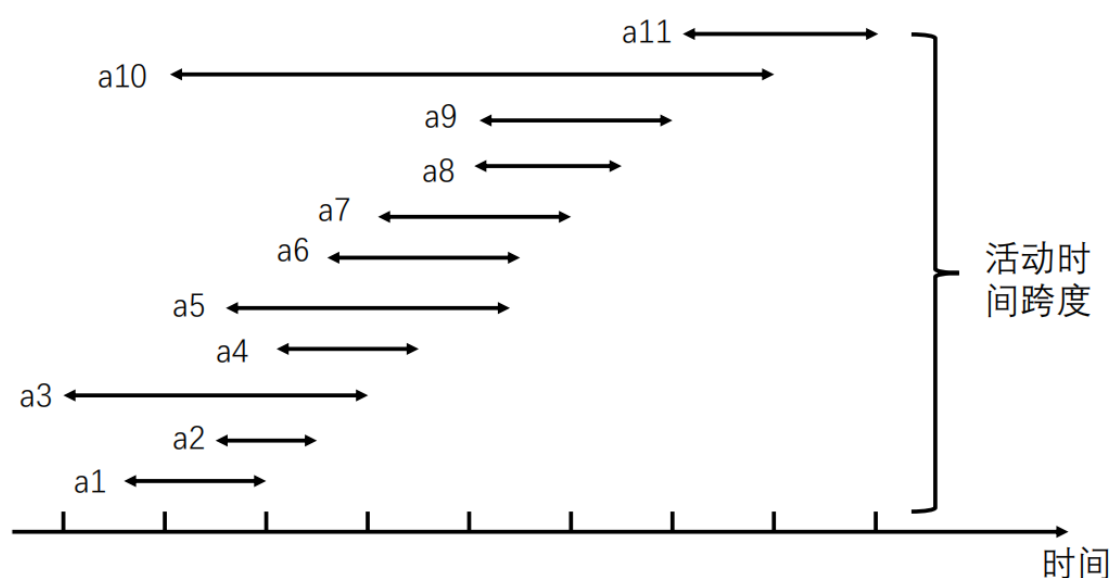
实验 4-2: 活动安排(贪心算法)

1. 问题描述, 题目定义。

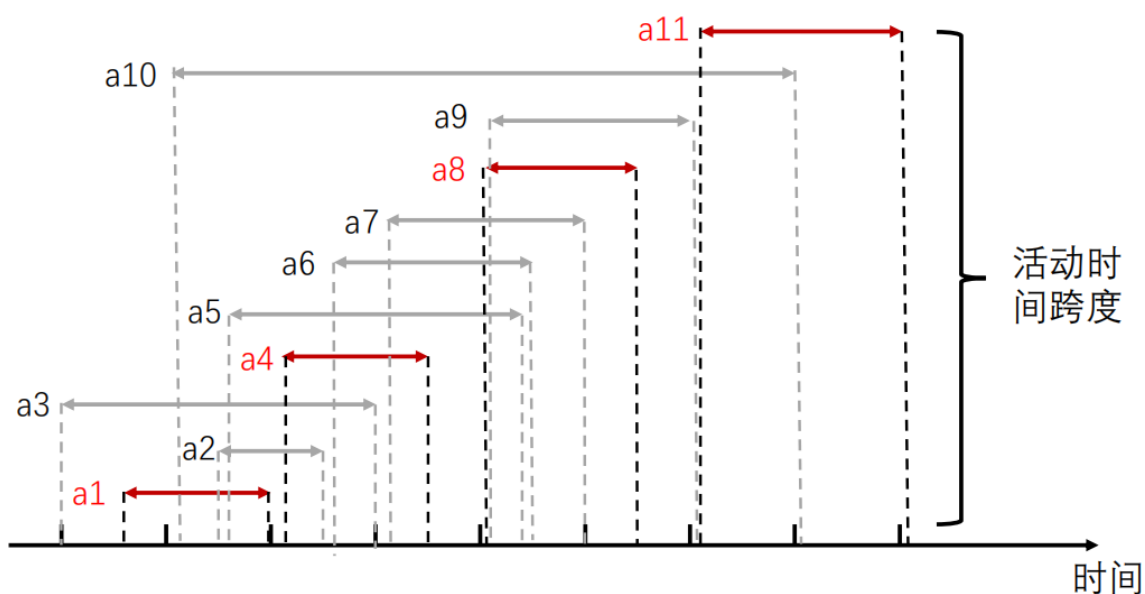
输入 n 个活动组成的集合 $A=\{a_1, a_2, \dots, a_n\}$, 每个活动 a_i 的开始时间 s_i 和结束时间 f_i 。找出活动集合 A 的子集 A' , 令 $\max|A'|$ s.t. 所有 a_i, a_j 满足 $s_i \leq f_j$ 或 $s_j \leq f_i$ 。

2. 算法实例, 实例演示程序过程。

输入: $s = \{1, 3, 0, 5, 3, 5, 6, 8, 8, 2, 12\}$; $f = \{4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14\}$; 以 a_1 至 a_{11} 命名。
其活动分布如下图所示:



输出:



选择的活动如上图所示。分别选择 a_1 , a_4 , a_8 , a_{11} 。活动数为 4

3. 程序代码，完整可运行程序，并加注释。

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int arrangeGreedy(vector<int> s, vector<int> f, int * a);

int main() {
    vector<int> s = { 1, 3, 0, 5, 3, 5, 6, 8, 8, 2, 12};
    vector<int> f = { 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14}; //已按结束时间排序
    int *a = new int[s.size()];
    int ans;
    ans = arrangeGreedy(s, f, a);
    cout << "选择的活动为: " << " ";
    for (int j = 0; j < s.size(); j++) { // 输出所选活动
        if (a[j] == 1)
            cout << j + 1 << ", ";
    }
    cout << endl;
    cout << "选择的活动数为: " << ans << endl;
    system("PAUSE");
    return 0;
}

int arrangeGreedy(vector<int> s, vector<int> f, int * a)
{
    int n = s.size(), j = 1, i, count = 1;
    a[0] = 1; //选择第 1 早结束的活动
    for (i = 1; i < n; i++)
    {
        if (s[i] >= f[j])
        {
            a[i] = 1; //选择 ai
            j = i;
            count++;
        }
        else
            a[i] = 0;
    }
    return count;
}
```

```
}
```

4. 程序验证，要求有不同的输入、输出，并截图。

```
vector<int> s = { 1, 3, 0, 5, 3, 5, 6, 8, 8, 2, 12};  
vector<int> f = { 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14}; // 已按结束时间排序
```

E:\算法设计与分析\4\实验4-2: 活动安排(贪心算法).exe

```
选择的活动为: 1, 4, 8, 11,  
选择的活动数为: 4  
请按任意键继续. . .
```

```
vector<int> s = { 1, 3, 0, 5, 3, 7, 6, 8, 5, 2, 12};  
vector<int> f = { 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14}; // 已按结束时间排序
```

E:\算法设计与分析\4\实验4-2: 活动安排(贪心算法).exe

```
选择的活动为: 1, 4, 6, 11,  
选择的活动数为: 4  
请按任意键继续. . .
```

5. 算法分析，时间复杂度与空间复杂度。

本算法的时间复开销来源于两部分，一部分是对结束时间序列进行排序，时间复杂度为 $O(n\log n)$ ，第二部分是贪心策略选择活动，时间复杂度为 $O(n)$ ，综合，算法的时间复杂度为 $O(n\log n)$ 。但是由于本次题目给定的结束时间序列已经默认完成了排序，所以此次题目的算法时间复杂度为 $O(n)$ 。空间开销只有一个长度为 n 的一维数组，用来记录哪个活动被选中了。 n 为给的总活动数量。所以空间复杂度为 $O(n)$ 。

时间复杂度为 $O(n)$

空间复杂度为 $O(n)$

实验 4-3: 最大子段和(贪心算法)

1. 问题描述, 题目定义。

给定 n 个数(可以为负数)的序列 $(a_1, a_2, \dots, a_n)$, 找到一个具有最大和的连续子数组 (子数组最少包含一个元素), 返回其最大和。

2. 算法实例, 实例演示程序过程。

$$\max\{0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k\}$$

实例 1: 输入: $(-2, 11, -4, 13, -5, -2)$

输出: 20 // $a_2 + a_3 + a_4 = 20$

实例 2: 输入: $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$

输出: 6 // $a_4 + a_5 + a_6 + a_7 = 6$ 。

3. 程序代码, 完整可运行程序, 并加注释。

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include "stdlib.h"
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

class Solution {
public:
    int maxSubArray(vector<int>& nums, int *begin, int *end) {
        int result = 0;
        int sum = 0;

        *begin = 0;
        for (int i = 0; i < nums.size(); i++) {
            sum += nums[i];
            if (sum > result) { // 取区间累计的最大值 (相当于不断确定最大子序终止位置)
                result = sum;
                *end = i;
            }
            if (sum <= 0) {
                sum = 0; // 相当于重置最大子序起始位置, 因为遇到负数一定是拉低
```

总和

```
        *begin = i + 1;
    }
}
return result;
}
};

int main() {
    int ans;
    Solution solution;

    vector<int> a = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    int begin=0, end=0;

    ans=solution.maxSubArray(a,&begin,&end);
    cout << "最大和为: ";
    cout << ans << endl;
    cout << "起止位置为: ";
    cout << begin << "," << end << endl;
    system("PAUSE");
    return 0;
}
```

4. 程序验证，要求有不同的输入、输出，并截图。

```
vector<int> a = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
int begin=0, end=0;
```

E:\算法设计与分析\4\实验4-3: 最大子段和(贪心算法).exe

```
最大和为: 6
起止位置为: 3, 6
请按任意键继续. . .
```

```
vector<int> a = {-2, 5, -3, 4, -9, 2, 7, -5, 1};
int begin=0, end=0;
```

E:\算法设计与分析\4\实验4-3: 最大子段和(贪心算法).exe

```
最大和为: 9
起止位置为: 5, 6
请按任意键继续. . .
```


5. 算法分析，时间复杂度与空间复杂度。

本算法使用了贪心策略来对序列进行选择，用了一个循环，所以时间复杂度为 $O(n)$ ，在算法中并没有使用数组记录，而只是使用 `begin`、`end` 两个变量来记录范围，所以空间开销为常量级 $O(1)$ ，相比于动态规划，空间开销小了，因为不用记录 `dp` 数组。

时间复杂度： $O(n)$

空间复杂度： $O(1)$

结果分析和总结：

贪心算法是通过一系列选择得到问题的解。其所做出的每一个选择都是当前状态下的局部最好选择，即贪心选择。一般来说，选择贪心算法，我们都要进行严格证明，证明贪心解不劣于最优解。这是比较麻烦的一点，有时候，我们并没有办法进行严格的证明。所以，贪心算法并不总能得到问题的最优解。

贪心算法并没有固定的解题步骤，很难寻找到所有问题统一的解法，需要多加练习、思考，也需要一定的数学思考能力。

本 次 上 机 成 绩	项目	操作	报告书写	出勤和课堂纪律	其他
	得分				
	成绩合计：				

评语：

教师签字：

日期：